
TECNOLOGÍAS CHAPINAS, S.A. | SISTEMA DE FACTURACIÓN

202402955 – Emily Maritza Tepeu Guacamaya

Resumen

Este proyecto consiste en el desarrollo de una solución integral para la gestión y facturación de servicios de infraestructura en la nube, implementando un API RESTful utilizando Python y Flask. La solución permite administrar recursos tecnológicos, categorías de configuración, clientes e instancias, procesando consumos de recursos y generando facturas periódicas. Se aplicaron principios de programación orientada a objetos (POO) para construir un software modular y escalable, junto con el uso de bases de datos XML para el almacenamiento persistente. El sistema incluye un frontend en Django para la interacción con el usuario y la generación de reportes en PDF. Los impactos técnicos incluyen la automatización de procesos de facturación y la capacidad de analizar ventas por categorías y recursos, mejorando la eficiencia operativa de la empresa.

Palabras clave

Facturación en la nube, API RESTful, Python, Flask, Django

Abstract

This project involves the development of a comprehensive solution for managing and billing cloud infrastructure services, implementing a RESTful API using Python and Flask. The solution allows managing technological resources, configuration categories, clients, and instances, processing resource consumption, and generating periodic invoices. Object-oriented programming (OOP) principles were applied to build modular and scalable software, along with the use of XML databases for persistent storage. The system includes a Django frontend for user interaction and PDF report generation. Technical impacts include the automation of billing processes and the ability to analyze sales by categories and resources, improving the company's operational efficiency.

Keywords

Cloud billing, RESTful API, Python, Flask, Django

Introducción

La computación en la nube ha revolucionado la forma en que las empresas gestionan sus recursos tecnológicos, ofreciendo escalabilidad y flexibilidad. Sin embargo, la facturación detallada de estos servicios representa un desafío técnico significativo. Este proyecto desarrolla un sistema backend-frontend para Tecnologías Chapinas, S.A., que automatiza la gestión y facturación de recursos en la nube. Utilizando Python con Flask para el API RESTful y Django para el frontend, el sistema procesa configuraciones de infraestructura, consumos de recursos y genera facturas periódicas. El propósito es demostrar cómo la programación orientada a objetos y el uso de bases de datos XML pueden crear soluciones robustas y mantenibles para problemas complejos de facturación en la nube.

Desarrollo del tema

a. Arquitectura del Sistema

El sistema sigue una arquitectura cliente-servidor donde el frontend (Django) interactúa con el backend (Flask) a través de APIs RESTful. Esta separación permite una clara división de responsabilidades: el frontend se encarga de la presentación y interacción con el usuario, mientras que el backend procesa la lógica de negocio y la persistencia de datos. El backend procesa mensajes XML para configurar recursos, categorías, clientes e instancias, almacenando toda la información en archivos XML que funcionan como base de datos. Esta arquitectura permite escalabilidad y mantenibilidad, ya que cada componente puede evolucionar independientemente.

b. Procesamiento de Datos XML y Expresiones Regulares

El sistema utiliza expresiones regulares para extraer fechas y validar NITs desde los mensajes XML de entrada. Por ejemplo, las fechas se extraen de cadenas de texto mediante patrones que identifican el formato dd/mm/yyyy, asegurando flexibilidad en la entrada de datos mientras se mantiene la integridad de la información. Para el NIT se implementó un patrón que verifica la estructura: sucesión de números seguida de un guion y un dígito de validación (0-9 o K). El procesamiento de XML se realiza mediante librerías como `xml.etree.ElementTree`, permitiendo un parsing eficiente de los mensajes de configuración y consumo.

c. Programación Orientada a Objetos y Diseño de Clases

Se implementaron clases como Recurso, Categoria, Configuracion, Cliente, Instancia y Factura, encapsulando la lógica de negocio y permitiendo reutilización de código. La herencia y polimorfismo facilitan la extensión del sistema para nuevos tipos de recursos o categorías. Cada clase tiene responsabilidades bien definidas: la clase Recurso maneja métricas y costos por hora; la clase Configuracion agrupa recursos; la clase Instancia gestiona el estado y consumo de recursos; y la clase Factura calcula montos y genera reportes.

d. Gestión de Consumos y Algoritmos de Facturación

El sistema calcula automáticamente los costos basados en el tiempo de uso de cada recurso, aplicando tarifas por hora. El proceso de facturación

agrupa consumos no facturados por cliente y genera facturas únicas con montos detallados por instancia y recurso. Se implementó un algoritmo que recorre todas las instancias activas de un cliente, calcula el consumo total por recurso y aplica las tarifas correspondientes. Para instancias canceladas, el sistema considera solo el consumo hasta la fecha de cancelación.

e. Framework Flask y Diseño de APIs RESTful

El backend utiliza Flask para exponer endpoints como /configurar, /consumir, /facturar y /reportes. Cada endpoint valida los datos de entrada, procesa la solicitud y retorna respuestas estructuradas en JSON. Se implementaron códigos de estado HTTP apropiados (200, 400, 500) para comunicar el resultado de las operaciones. La autenticación se maneja mediante tokens JWT para asegurar que solo usuarios autorizados puedan acceder a los endpoints.

f. Frontend Django y Patrón MVT

El frontend implementa el patrón Modelo-Vista-Template (MVT) de Django. Los modelos representan las entidades del sistema, las vistas procesan las solicitudes del usuario y los templates generan la interfaz HTML. Se implementaron formularios para la carga de archivos XML, tablas para mostrar datos existentes y gráficos para el análisis de ventas. La generación de PDFs se realiza mediante librerías como ReportLab, creando documentos profesionales con logos, tablas detalladas y gráficos estadísticos.

g. Persistencia de Datos y Validación con XSD

Aunque el sistema utiliza archivos XML como base de datos, se implementó un esquema XSD para validar la estructura de los datos almacenados. Esto asegura la integridad de la información y facilita la detección de corrupción en los archivos. Para operaciones complejas de consulta, se implementaron métodos de búsqueda y filtrado que simulan funcionalidades de bases de datos relacionales.

h. Manejo de Estados y Transiciones

Las instancias implementan un patrón de estado que controla las transiciones entre "Vigente" y "Cancelada". Cuando una instancia se cancela, se registra la fecha final y se detiene el cálculo de consumos. El sistema valida que las instancias canceladas tengan fecha final, manteniendo la consistencia de los datos.

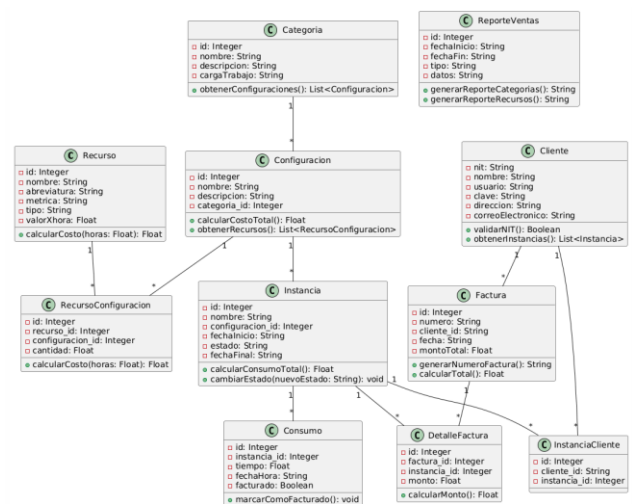


Figura 1. Diagrama de clases.

Fuente: Elaboración propia

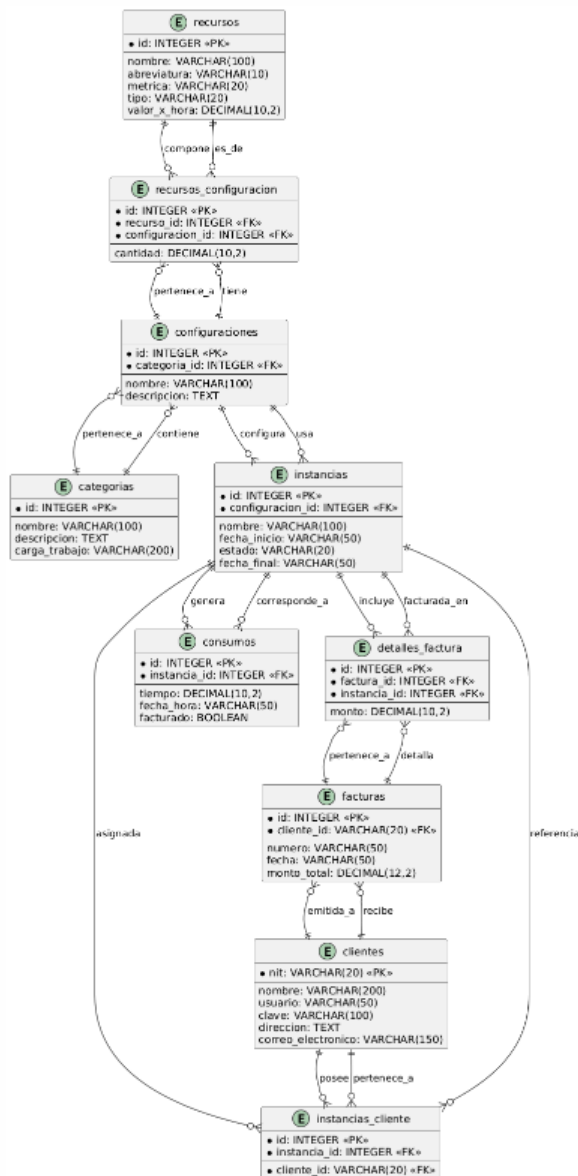


Figura 2. Modelo de datos

Fuente: Elaboración propia

extensible, mientras que el uso de XML como base de datos aseguró la persistencia de la información. Los reportes en PDF y el análisis de ventas proporcionan herramientas valiosas para la toma de decisiones. Como trabajo futuro, se podría integrar una base de datos relacional para mejorar el rendimiento con grandes volúmenes de datos.

Referencias bibliográficas

Flask Documentation. (2023). Flask Web Development. <https://flask.palletsprojects.com/>

Django Software Foundation. (2023). Django Documentation. <https://docs.djangoproject.com/>

Date, C. J. (1991). An Introduction to Database Systems. Addison-Wesley.

Lutz, M. (2013). Learning Python. O'Reilly Media.

Freedman, A. (2019). The Cloud Revolution. Pearson Education..

Conclusiones

El proyecto demuestra la viabilidad de desarrollar un sistema integral de facturación para servicios en la nube utilizando tecnologías modernas como Python, Flask y Django. La implementación de POO permitió crear un código modular y

Extensión: de cuatro a siete páginas como máximo

Adicionalmente, se pueden agregar apéndices con modelos, tablas, etc. Que complementan el contenido del trabajo.